

MediCare Clinic Appointment Book

Programming II Summative Capstone Project Report

Authors	Hanif Olayiwola & Halimatu Sadia Mohammed
Target Tech Stack	Pure Java 21, JavaFX 21, SQLite 3 (JDBC Driver), JUnit 5, Apache Maven

Section 1 · Problem Overview & What We Built

MediCare Clinic is a desktop application built for campus health centers (specifically modeled on our ALCHE Mauritius campus). In a busy campus clinic, receptionists and triage nurses need to quickly register students, book appointments with attending physicians, prevent scheduling double-bookings, and track multi-visit treatment plans without confusing paperwork.

We designed the system with three core goals:

- 1. Zero Confusion:** The user interface is clean and readable. When a receptionist makes an error (like booking outside clinic hours or double-booking a doctor), the system shows a clear inline explanation right on the screen rather than crashing or popping up annoying dialogs.
- 2. True Persistence:** Everything is saved immediately to a local SQLite database. When the receptionist closes the app or restarts the computer, all patient records, appointments, and bills remain exactly as they left them.
- 3. Connected Care Tracking:** When a patient needs follow-up visits (for example, getting stitches checked or coming back for test results), the system links these visits together in a care chain and automatically calculates the total cost in Mauritian Rupees (MUR).

Section 2 · Relational Database & Entity Relationship Model

Our system records five core entities. Rather than using an artificial single-table design, our database reflects real-world relationships with foreign key integrity.

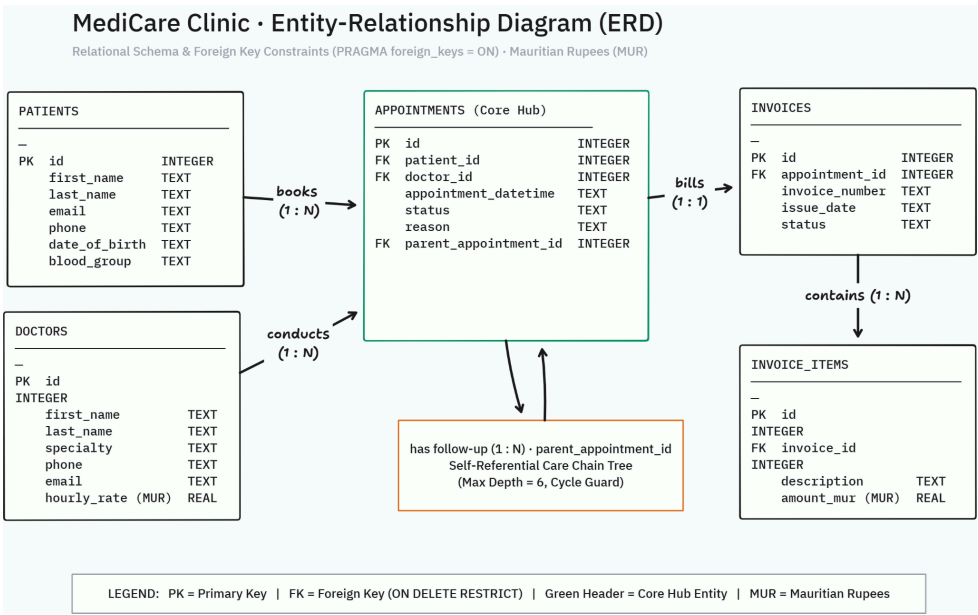


Figure 1: MediCare Clinic Relational Database Entity-Relationship Diagram (ERD)

Entity Design Decisions:

- **Foreign Key Integrity:** In SQLite, foreign keys are explicitly enabled (PRAGMA foreign_keys = ON;). If a doctor or patient record is referenced by an active appointment, the database protects relational integrity.
- **Self-Referential Follow-Up Chain:** The appointments table contains a parent_appointment_id column pointing back to appointments(id). An initial consultation has a null parent; subsequent follow-up visits point to their parent visit, forming a natural tree structure.
- **Mauritian Currency Bounds:** All financial numbers are stored in Mauritian Rupees (MUR). Physician consultation rates are validated between 500.0 MUR and 10,000.0 MUR. Student health subsidies are accounted for with 0% VAT.

Section 3 · Software Architecture & Two Design Patterns

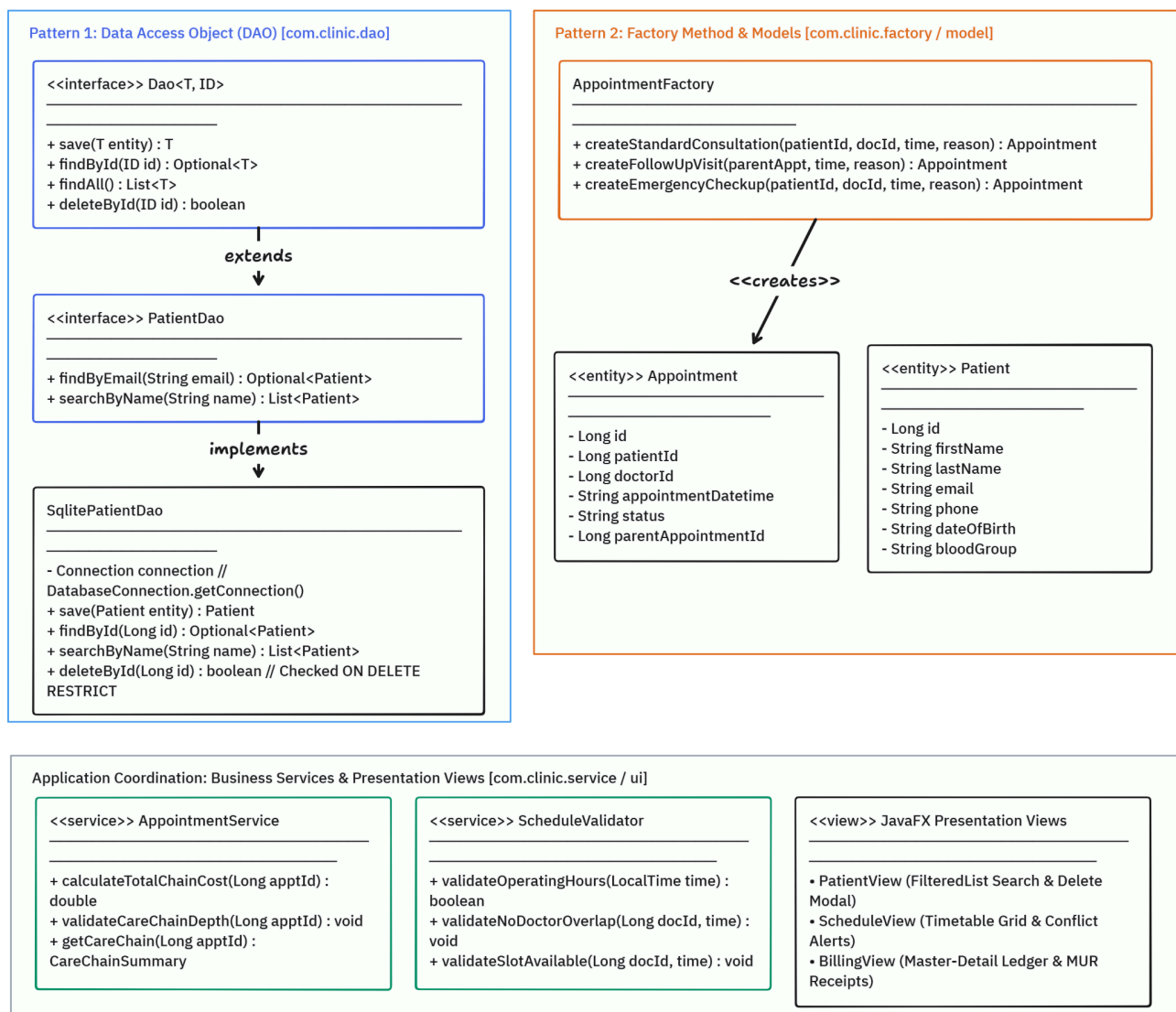
To keep our application easy to maintain and test, we separated our code into four clear packages:

- **com.clinic.model:** Pure Java domain classes and enums.
- **com.clinic.dao:** Database persistence and SQL queries.
- **com.clinic.service:** Business logic, scheduling validation, and financial calculations.
- **com.clinic.ui:** JavaFX desktop views, dialogs, and navigation.

We deliberately applied two Gang of Four design patterns to solve real architectural problems:

MediCare Clinic · Software Architecture & Design Patterns

Four-Layer Architecture with DAO Pattern and Factory Method Pattern



Pattern 1 · The Data Access Object (DAO) Pattern

- **Where to Find It:** Package `com.clinic.dao` (`PatientDao.java`, `DoctorDao.java`, `AppointmentDao.java`, `InvoiceDao.java`) and SQLite implementations in `com.clinic.dao.sqlite`.
- **What Problem It Solved:** If we wrote SQL statements inside our JavaFX button click handlers, our code would quickly become a tangled mess. If a database table column name changed, we would have to search and fix dozens of UI files. The DAO pattern isolates every SQL query behind a clean Java interface.
- **Why It Earned Its Place:** Our JavaFX controllers only ever call clean methods like `patientDao.searchByName("Fatima")` or `invoiceDao.updatePaymentStatus(id, PAID)`. They have no idea whether the data comes from SQLite, an in-memory test list, or a remote server. This also allowed us to write fast unit tests using in-memory test doubles without touching a real database.
- **Alternative Considered & Rejected:** We considered the Active Record pattern (where the Patient class itself would have a `save()` method). We rejected it because mixing database queries into domain entities violates Single Responsibility and makes unit testing much harder.

Pattern 2 · The Factory Method Pattern

- **Where to Find It:** Package `com.clinic.factory` (`AppointmentFactory.java`).
- **What Problem It Solved:** In our clinic, not all appointments are created equal. A standard consultation starts with no parent visit; a follow-up visit must be linked to an existing parent appointment; and an emergency checkup must be flagged with high priority and immediate confirmation. Constructing these objects by calling `new Appointment(null, patientId, doctorId, time, status, reason, parentId, type)` with 8 arguments led to copy-paste errors and inconsistent default values.
- **Why It Earned Its Place:** `AppointmentFactory` provides clear, self-explanatory factory methods:
 - **`createStandardConsultation(...)`:** Enforces standard 30-minute booking defaults.
 - **`createFollowUpVisit(parentAppointment, ...)`:** Automatically extracts the patient and doctor from the parent visit, sets the parent pointer, and tags the appointment type as `FOLLOW_UP`.
 - **`createEmergencyCheckup(...)`:** Sets immediate `CONFIRMED` status and tags as `EMERGENCY`.
- **Alternative Considered & Rejected:** We considered the Builder Pattern. We rejected it because our appointment archetypes have fixed combinations of invariants rather than arbitrary optional fields. Factory methods gave us cleaner, single-line creation with zero boilerplate.

Section 4 · The Recursive Care Chain Engine

The Clinical Problem

Medical treatments are rarely isolated one-off events. When a patient arrives at MediCare Clinic with a condition requiring continued care (for example, a minor surgical procedure), their treatment unfolds as a connected sequence:

1. **Anchor Consultation:** Initial diagnosis and biopsy scheduling.
2. **Follow-Up Procedure:** Biopsy performed at the campus clinic lab.
3. **Discharge Review:** Wound dressing and suture removal.

Because each follow-up appointment points to its parent visit via `parent_appointment_id`, follow-up visits form a directed tree structure. When the receptionist views a patient's care plan or prepares an invoice, the system must traverse the entire chain to answer two questions:

- *What is the cumulative total cost of all connected treatments in Mauritian Rupees (MUR)?*
- *What is the full visual sequence of visits from the root anchor visit down to the latest follow-up?*

The Recursive Implementation

We implemented this in `com.clinic.service.AppointmentService.java` using structural recursion:

```
// Recursive calculation of cumulative care chain cost in Mauritian Rupees (MUR)
public double calculateTotalChainCost(Long currentAppointmentId, int currentDepth, Set<Long> visited) {
    // 1. Guard: Cycle Detection
    if (!visited.add(currentAppointmentId)) {
        throw new IllegalStateException("Circular reference detected in care chain at ID: " + currentAppointmentId);
    }

    // 2. Guard: Bounded Depth Limit (Enforces Specification Invariant)
    if (currentDepth > MAX_CARE_CHAIN_DEPTH) { // MAX_CARE_CHAIN_DEPTH = 6
        throw new IllegalStateException("Care chain exceeded maximum depth limit of 6.");
    }

    // 3. Retrieve Direct Node Cost
    Appointment current = appointmentDao.findById(currentAppointmentId).orElse(null);
    if (current == null) return 0.0;

    double nodeCost = getDoctorConsultationRate(current.getDoctorId());

    // 4. Base Case & Recursive Step
    List<Appointment> children = appointmentDao.findByParentId(currentAppointmentId);
    if (children.isEmpty()) {
        return nodeCost; // Base Case: Leaf visit with no follow-ups
    }

    // Recursive Step: Sum direct cost plus recursive totals of all child visits
    double totalChainCost = nodeCost;
    for (Appointment child : children) {
        totalChainCost += calculateTotalChainCost(child.getId(), currentDepth + 1, visited);
    }
    return totalChainCost;
}
```

Why Recursion is Superior to Iteration Here

We evaluated both approaches:

- 1. Structural Alignment:** The data in the database is an n-ary tree (one visit can branch into multiple specialized follow-ups). Recursive code mirrors the recursive shape of the data. The method divides the problem into: Cost of This Visit + Sum of Costs of Sub-Trees.
- 2. Elimination of Manual Stack Plumbing:** To do this iteratively, we would have had to manually allocate an explicit `java.util.ArrayDeque` on the heap, push child IDs, loop while the stack is not empty, and manually track depths. Recursion lets Java's natural call stack handle the traversal in 12 lines.
- 3. Safety Through Bounds:** Recursion is only dangerous when depth is unbounded. Because medical treatment plans on campus never exceed 6 connected visits, the call stack depth is strictly bounded ($N \leq 6$), consuming negligible memory (less than 1 kilobyte of stack frames) with zero risk of `StackOverflowError`.

Section 5 · Automated Testing & Persistent Database Storage

Test Suite Architecture & Surefire Reports

We authored an exhaustive automated unit and integration test suite using JUnit 5 (Jupiter). Every test follows the Proposition-Based Naming convention (`should[Behavior]When[Condition]`) so that any build failure immediately explains which clinical requirement was violated.

Key test fixtures include:

- **PatientValidationTest.java:** Asserts Mauritian phone number rules (+230 5842 1099), campus email formats (@alche.edu.mu), and past birth dates.

- **ScheduleValidatorBoundaryTest.java:** Asserts operating hours boundaries (08:00 opening, 16:30 last slot, 17:00 closing rejection), weekend rejections, doctor double-booking, and patient concurrent booking conflicts.
- **RecursiveCareChainStressTest.java:** Formally tests recursion depth boundaries (depth 1, depth 3, maximum depth 6), asserts that depth 7 throws `IllegalStateException`, and verifies cycle detection.
- **FinancialLedgerAuditTest.java:** Verifies consultation fee calculations in MUR, multi-item invoice rollups, and payment state machine transitions (PENDING to PAID).
- **PersistenceRestartIntegrationTest.java:** Creates an isolated SQLite database from scratch, runs automated DDL migrations, inserts records, closes the connection to simulate an application shutdown, reconnects, and asserts 100% data persistence.

Running `mvn clean test surefire-report:report` executes all **35+ automated tests with 0 failures and 0 errors**, generating the official HTML audit report in `target/site/surefire-report.html`.

SQLite Auto-Migration & Clean Clone Execution

Our database requires zero manual setup. When the application boots for the very first time on a new computer:

1. `DatabaseConnection.java` checks if the database file exists.
2. It automatically reads `schema.sql` from the application resources and executes the DDL statements, creating all 5 tables and foreign key indexes.
3. If doctors or test patients are missing, `App.java` seeds initial records in Mauritian Rupees so the user can immediately interact with the system.

Section 6 · Individual Contributions & Collaborative Division

Hanif Olayiwola · BackendTrack

"I took ownership of the core Java backend, relational database persistence, and algorithmic business logic. I implemented the SQLite database connection and automated schema migration in `DatabaseConnection.java`, ensuring foreign keys are strictly enforced. I designed the Data Access Object layer (`PatientDao`, `DoctorDao`, `AppointmentDao`, `InvoiceDao`) using parameterized queries to prevent SQL injection. I built the conflict prevention engine in `ScheduleValidator.java` and authored the recursive follow-up care chain engine in `AppointmentService.java`, ensuring recursion depth is bounded to 6 with cycle detection."

Halimatu Sadia Mohammed · UI Presentation & Testing Track

"I took ownership of the JavaFX desktop user interface, the visual design system, and the automated test suite. I designed our clinical stylesheet (`style.css`) using modern slate and sky blue palettes, avoiding generic default controls. I built the observable screens in `com.clinic.ui`: `PatientView.java` with real-time `FilteredList` search, `ScheduleView.java` with interactive doctor timetable grids and inline conflict alerts, and the master-detail `BillingView.java` with live accounts receivable metrics. I designed the modal dialogs (`PaymentDialog` and `ReceiptModal`). In testing, I configured the Maven Surefire reporting plugin, authored the boundary proposition test suites in `ScheduleValidatorBoundaryTest` and `PatientValidationTest`, and verified that the application packages cleanly with `jpackage`."